

CRM Scoring

AI Lead Generation & Scoring Agent

Doc version **1.0.0** Status **Draft – engineering review** Date **September 1, 2026**

Source **backend/, frontend/, licensing/**

Introduction

CRM Scoring (shipped as the *AI Lead Generation & Scoring Agent*) is a self-hostable B2B lead-scoring pipeline. A user uploads a CRM export; every row is enriched with firmographic and behavioral signals, scored by a hybrid rule-based + Claude model, ranked into **HOT** **WARM** **COLD** buckets, and — for the hot ones — can be turned into a drafted outreach message and pushed into Salesforce, all from one screen.

Every third-party integration (Claude, Apollo enrichment, Salesforce, Slack) auto-detects its own credentials: set a key and it makes the real call; leave it unset and it runs on deterministic mock data. The application is licensed and sold as self-hosted software, verified fully offline against a signed key — no phone-home required.

60 / 40

rule-based vs. LLM blend into the combined score

0 keys

required to run the full pipeline end to end, on mocks

1 process

one port in production — FastAPI serves the built React app

Purpose

This document defines the product and technical requirements for CRM Scoring: system architecture, functional requirements, the enrichment/scoring pipeline, data model, tenant/licensing model, and API contract — a single reference for engineering, sales, and support.

Product Overview

CRM Scoring is not a chat product — there is no conversational surface and no end-user safety-crisis workflow. It is a batch/on-demand pipeline sold two ways:

- **Direct self-hosted sale** — a buyer runs the app themselves against their own data, gated by an offline-verifiable license key, with a time-and upload-boxed free trial ahead of purchase.
- **Seller-hosted multi-tenant deployment** — one operator runs a single shared instance and provisions isolated tenants (their own customers) either by hand or through self-serve signup; those tenants are never gated by the deployment owner's own license.

A second, separate capability — churn-risk scoring for subscriber/customer exports — ships in the same app as a free, unlicensed evaluation tool; it uses a different data shape than a B2B lead and is not the thing the product is sold as.

Tech Stack

LAYER	TECHNOLOGY
Frontend	React 19 + TypeScript + Vite, Tailwind CSS
Backend API	FastAPI (Python 3.12) — core REST API for ingestion, scoring, and actions
Database	SQLite (zero-config default, <code>backend/data/app.db</code>) or PostgreSQL via <code>DATABASE_URL</code> — parameterized SQL, no ORM
LLM provider	Anthropic Claude (<code>ANTHROPIC_MODEL</code> , default <code>claude-sonnet-5</code>) — scoring assessment and outreach drafting
Enrichment	Apollo.io organization-enrichment API, with a deterministic seeded mock fallback
CRM integration	Salesforce (<code>simple-salesforce</code>) — auto-provisions its own custom Lead fields on first push
Alerts	Slack (<code>slack_sdk</code>), with an SMTP path available for email as an alternative channel
Payments / licensing	Paddle (overlay checkout) and Polar (redirect checkout), both merchant-of-record; offline Ed25519-signed license keys via PyNaCl
Rate limiting	<code>slowapi</code> (the <code>limits</code> library for FastAPI)
Auth	Tenant API key (<code>Authorization: Bearer</code>); PBKDF2-HMAC-SHA256 password hashing for self-serve accounts

System Architecture (High-Level)

Requests flow from the React client through a FastAPI gateway into the lead-intelligence pipeline, and out to SQLite/Postgres for persistence. In production, one process serves both the API and the built frontend:

LAYER	RESPONSIBILITY
React SPA	Upload panel, ranked/filterable leads table, lead detail drawer (firmographics, score breakdown, LLM rationale, outreach draft, CRM push), license/billing banner, tenant switcher
FastAPI gateway	CORS allowlist, security headers, per-endpoint rate limiting, tenant resolution from the <code>Authorization</code> header, Pydantic request/response validation
Upload validation	File-type allowlist, size cap, row cap — enforced before a file is ever handed to a parser
Lead intelligence pipeline	Ingestion → Enrichment → Hybrid Scoring → Actions (outreach draft, CRM push, Slack alert)
Licensing layer	Offline Ed25519 license verification; deployment-wide trial clock and upload counter
Storage	SQLite or Postgres — <code>leads</code> , <code>alerts</code> , <code>tenants</code> , <code>password_resets</code> , <code>app_meta</code> , every row tenant-scoped

Two run modes, one codebase. In dev, Vite serves the frontend on :5000 and proxies `/api/*` to FastAPI on :8081. In production (`run-prod.sh` , or the Docker image), FastAPI itself serves the built `frontend/dist` — a catch-all route registered last, behind every `/api/*` router, so a hard refresh on a client-side route like `/purchase-complete` still resolves correctly. One process, one port.

Core Functional Requirements

Lead Ingestion & Upload

- Leads are ingested via `POST /api/leads/upload` — a CSV or XLSX file containing at minimum a company-name and a domain/website column.
- Common CRM export headers are normalized through a column-alias map (`company / account_name` → `company_name` ; `website / domain` → `domain` ; `job_title / title` → `contact_title` ; and so on) so the file doesn't need to match the internal schema exactly.
- A file missing both a company-name and a domain column is rejected with `400` before any row is processed.
- Every row is minted a fresh 12-character identifier server-side; the client never assigns lead IDs.
- Re-uploading a file containing a domain that already exists for the tenant **updates** that lead rather than duplicating it — leads are keyed by `(tenant_id, domain)` at the storage layer.

Multi-Tenant Access Model

- A request with no `Authorization` header resolves to a sentinel **default tenant** — the zero-config, single self-hosted-buyer case. No signup, no key, nothing to configure.
- A request with `Authorization: Bearer <api-key>` resolves to whichever tenant hashes to that key. An unrecognized key is rejected with `401` — it never silently falls back to the default tenant, so a typo'd key can't leak into (or read) someone else's data.
- Every lead and alert row carries a `tenant_id`, and every storage read/write is scoped by it — a tenant cannot enumerate or read another tenant's leads by guessing an ID.
- Two ways to provision a tenant: `backend/scripts/create_tenant.py` (manual, run by the seller) and self-serve signup (`/api/accounts/signup`).

Self-Serve Signup, Login & Password Reset

- **Signup** (name, email, password) provisions a tenant and returns a freshly minted API key *once* — only its SHA-256 hash is ever stored, so it cannot be recovered later, only rotated.
- Password rule: 8–200 characters, at least one uppercase letter, one lowercase letter, one digit, and one symbol. The first unmet rule is reported by name, not a generic "invalid password."
- **Login** hands back a freshly rotated API key rather than a session token — this app has exactly one credential type. A real trade-off: logging in on a second device invalidates the key issued to the first, since there is one valid key per tenant, not one per device.
- **Forgot / reset password** issues a single-use, time-boxed token (30 minutes by default, `PASSWORD_RESET_TTL_MINUTES`), consumed and deleted on first use whether or not it was still valid. The response is identical whether or not the email has an account, so the endpoint can't be used to enumerate registered emails.

- Signup, login, and forgot-password are all rate-limited (`RATE_LIMIT_AUTH` , default **5/minute**) — classic brute-force/enumeration/spam targets.

API & Security Controls

CONTROL	IMPLEMENTATION
Rate limiting	<code>slowapi</code> — 100/min default globally, 10/min on upload/outreach/CRM-push/churn-upload, 5/min on signup/login/forgot-password
CORS	Explicit origin allowlist (<code>CORS_ALLOWED_ORIGINS</code>), <code>GET</code> / <code>POST</code> only, a specific header allowlist — never wildcarded
Security headers	<code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> , HSTS (HTTPS requests only), a CSP scoped to what Paddle/Polar checkout needs
HTTPS	<code>FORCE_HTTPS</code> redirects at the app level for a self-hosted deploy with no proxy in front doing it; off by default since most managed hosts already handle this
Proxy trust	<code>TRUST_PROXY_HEADERS</code> — only enabled when a trusted reverse proxy genuinely sits in front, otherwise a client could spoof its IP and dodge rate limits
SQL safety	Every query is parameterized (<code>?</code> placeholders) — nothing from an upload or request body is ever interpolated into SQL
Password storage	PBKDF2-HMAC-SHA256, random salt per user, 600,000 iterations — OWASP's 2023 minimum

Validation & Degradation Guardrails

CRM Scoring has no conversational safety surface, so its guardrails serve a different purpose than a chat product's: keep a malformed upload from burning resources, and keep a missing or misbehaving third-party integration from breaking the pipeline for anyone.

Inbound – Before a File Is Parsed

- **File-type allowlist** — only `.csv` , `.xlsx` , `.xls` are accepted; anything else is rejected with `400` before the bytes are ever handed to a parser.
- **Size cap** — `MAX_UPLOAD_SIZE_MB` (default 10 MB) rejects an oversized file with `413` before parsing.
- **Row cap** — `MAX_UPLOAD_ROWS` (default 50,000) rejects an oversized file with `400` , independent of any trial limit.
- **Field constraints** — every Pydantic model caps freeform text length and rejects negative numeric fields, so a spreadsheet cell with megabytes of text or a negative revenue figure can't distort a stored row.
- **Literal types** — the outreach channel, CRM target, and billing interval are all `Literal[...]` types rather than bare strings, since only a fixed set of values is ever valid.

Degrade, Don't Break

Instead of an "outbound" content guardrail, this product's equivalent principle is that a missing or failing third-party integration degrades a single lead or response — never the whole request:

INTEGRATION	NO CREDENTIALS CONFIGURED	LIVE CALL FAILS
Enrichment (Apollo)	Deterministic mock, seeded by a SHA-256 hash of the domain	Falls back to the same mock for that one lead
Scoring / outreach (Claude)	Mock rationale/draft, explicitly labeled and pointing at <code>ANTHROPIC_API_KEY</code>	Falls back to the mock, with the error appended to the text
CRM push (Salesforce)	Simulated push — logged, response says so explicitly	Falls back to the simulated push, response explains why
Alerts (Slack)	Logged locally and stored for the alerts panel	Logged locally and stored, with a warning logged

Every mock is deterministic (seeded by domain), so results are stable across repeated runs during local evaluation — and every mock text field says so explicitly, so a buyer can never mistake mock output for a live call.

Lead Intelligence Pipeline

Every uploaded row moves through four stages, each isolated behind a single function so the live/mock switch for one stage never touches the others.

Stage 1 – Ingestion

`services/ingestion.py` parses the CSV/XLSX with pandas, normalizes column names (lower-cased, spaces to underscores, aliases applied), and constructs one `Lead` record per row. Blank/NaN cells become `None` rather than the string `"nan"`.

Stage 2 – Enrichment

`services/enrichment.py` fills in whatever firmographic fields the source file didn't already supply:

- **Live path** — with `APOLLO_API_KEY` set, calls Apollo's organization-enrichment endpoint (retried up to 3 times with exponential backoff) for industry, employee count, revenue, geography, and hiring/tech-stack signals.
- **Mock path** — with no key (or on any live-call failure), generates industry/size/revenue/geography/tech-stack/hiring values from a random generator seeded by the domain's SHA-256 hash, so the same domain always enriches identically across runs.
- Output is a single normalized `EnrichedLead` record, tagging `enrichment_source` as `"apollo"` or `"mock"`.

Stage 3 – Hybrid Scoring

`services/scoring.py` blends two independent layers into one `combined_score`:

RULE-BASED LAYER

Seven weighted criteria are scored against a configurable Ideal Customer Profile (`ICP`), each criterion producing a 0–1 fit ratio multiplied by its weight; the weights sum to 100 and form the transparent `fit_score`:

CRITERION	WEIGHT	HOW IT'S COMPUTED
Industry match	20	Full credit if <code>industry</code> is in the ICP's target industry list, else 0.25
Company size fit	20	Band-fit against the ICP's employee-count range (1.0 inside the band, decaying linearly the further outside it)
Revenue fit	15	Same band-fit logic against the ICP's revenue range
Tech stack match	15	Fraction of the ICP's target tech stack found in the lead's own stack
Geography fit	10	Full credit inside the ICP's target geographies, else 0.2
Title seniority	10	Full credit if the contact title contains a decision-maker keyword (CEO, VP, Director, "head of", ...), else 0.3
Hiring signal	10	Full credit if the company is currently hiring, else 0

The default ICP targets SaaS / Fintech / Healthcare Tech / E-commerce companies, 50–1,500 employees, \$5M–\$250M revenue, a Salesforce/HubSpot/AWS/Snowflake/Stripe-shaped tech stack, in the US/Canada/UK — all editable in one config block to match whatever business a given deployment is generating leads for.

LLM LAYER

Claude receives the enriched profile *and* the rule-based breakdown, and returns a qualitative `conversion_likelihood` (0–100) plus a one- or two-sentence rationale — catching context the rules structurally can't, such as recent funding news or a hiring surge. The prompt requires a JSON-only response; a malformed or failed response falls back to a mock assessment that jitters the rule-based `fit_score` by up to ±8 points (seeded by domain hash) and lists which qualitative signals it noticed.

BLEND & BUCKET

SCORING.PY

```
combined_score = 0.6 * fit_score + 0.4 * conversion_likelihood

bucket = "hot"   if combined_score >= 75
        = "warm" if combined_score >= 50
        = "cold" otherwise
```

Stage 4 – Actions

Unlike ingestion/enrichment/scoring (which run automatically on every uploaded row), actions are per-lead and mostly on demand from the UI:

- **Outreach draft** — `POST /api/leads/{id}/outreach`, channel `email` or `linkedin`. Claude writes a short, personalized first-touch message referencing the lead's specific fit signals (tech stack, hiring activity, industry); the mock fallback does the same with template text.
- **CRM push** — `POST /api/leads/{id}/crm-push`. Writes `Fit_Score__c`, `Combined_Score__c`, `Lead_Bucket__c`, `LLM_Rationale__c`, and `Outreach_Draft__c` onto the matching Salesforce Lead (matched by website, created if none exists). Those custom fields are best-effort auto-created via Salesforce's Tooling API the first time a push runs.
- **Slack alert** — *not* on demand. Fires automatically, inline with scoring, the moment a lead's bucket becomes `"hot"` during an upload, so sales can act while the signal is fresh. A one-time preflight check (token validity, bot channel membership) logs a clear diagnostic if the integration is misconfigured, rather than surfacing a generic API error on the first real alert.

Churn-Risk Scoring (Separate Capability)

`POST /api/churn/upload` scores individual customer/subscriber rows (tenure, contract type, monthly charges, service add-ons, payment method) — a different data shape from a B2B lead, so it has its own model, its own endpoint, and its own weighted heuristic (not the LLM layer, not a trained model):

- Month-to-month contract: **+35** risk; one-year: **+15**; longer: **+0**
- Short tenure scales up to **+25** as tenure approaches 0 months (capped at 24 months for the ratio)
- High monthly charges scale up to **+15** across an \$18–\$120 reference band
- No tech support: **+8**; no online security add-on: **+7**
- Electronic-check payment method: **+10**
- Bucketed `high` (≥ 60), `medium` (≥ 30), or `low`

This endpoint is never license-gated and its results are not persisted — it exists so a prospect can evaluate scoring quality on this second data shape without needing a license, and results live only in the API response.

Data Architecture Overview

SQLite is the zero-config default (`backend/data/app.db`); setting `DATABASE_URL` to a `postgres(ql)://` connection string switches the same code path to Postgres for real concurrent-write support. Leads and alerts are stored as serialized JSON rows rather than a fully normalized schema — the shape is defined by the Pydantic models, so a new field never requires a migration.

TABLE / STORE	PURPOSE
leads	One row per (<code>tenant_id</code> , <code>domain</code>). Indexed columns (<code>tenant_id</code> , <code>domain</code> , <code>combined_score</code>) plus the full <code>ScoredLead</code> record as a JSON blob. Re-upserted, never duplicated, on re-upload.
alerts	Auto-incrementing hot-lead notifications, tenant-scoped, most-recent-first. One row is written every time a lead crosses into the <code>hot</code> bucket.
tenants	<code>id</code> , <code>name</code> , <code>api_key_hash</code> (SHA-256, unique), <code>created_at</code> , plus optional <code>email</code> (unique among non-null rows) and <code>password_hash</code> for self-serve accounts. Rows provisioned by <code>create_tenant.py</code> have no email/password of their own.
password_resets	<code>token_hash</code> (SHA-256, primary key), <code>tenant_id</code> , <code>expires_at</code> . Single-use — deleted the instant it's consumed, valid or not.
app_meta	Deployment-wide key/value store — <code>trial_started_at</code> , <code>trial_uploads_used</code> . Not tenant-scoped: this is one trial clock per self-hosted instance, and only ever applies to the default tenant.
issued_licenses.jsonl	Append-only log outside the database, written by the seller-side billing integration on every issued key — email, license key, issued-at timestamp. The fallback record of a purchase if email delivery fails.

Churn-risk results are the one exception: they are computed and returned in the API response but never written to any table — there is no `churn_customers` store.

API Communication Specification

Every route is `GET` or `POST`. Requests and responses are validated by FastAPI/Pydantic; errors use FastAPI's standard `{"detail": "<message>"}` body rather than a bespoke envelope.

Endpoints

METHOD & PATH	AUTH	PURPOSE
GET /api/health	None	Liveness check — <code>{"status": "ok"}</code>
GET /api/license	None	Deployment-wide license/trial status (not tenant-scoped)
POST /api/leads/upload	Optional Bearer	Upload a CSV/XLSX; ingests, enriches, scores, alerts, and stores every row
GET /api/leads	Optional Bearer	List the tenant's leads, ranked by <code>combined_score</code> descending
GET /api/leads/{lead_id}	Optional Bearer	Single lead; 404 if not found or not owned by this tenant
POST /api/leads/{lead_id}/outreach	Optional Bearer	Generate/regenerate an outreach draft for one lead
POST /api/leads/{lead_id}/crm-push	Optional Bearer	Push one lead's score/rationale/draft to Salesforce
GET /api/alerts	Optional Bearer	The tenant's stored hot-lead alerts, most recent first
POST /api/churn/upload	None	Score a churn/subscriber export; not persisted, not license-gated
POST /api/accounts/signup	None	Provision a new self-serve tenant + API key
POST /api/accounts/login	None	Verify credentials, rotate and return a fresh API key
POST /api/accounts/forgot-password	None	Issue a time-boxed reset token by email, if the account exists
POST /api/accounts/reset-password	None	Consume a reset token, set a new password, rotate the API key
GET /api/billing/config	None	Seller-side: Paddle client token/environment/price IDs, Polar

METHOD & PATH	AUTH	PURPOSE
		availability
POST /api/billing/webhook	Paddle signature	Seller-side: issues a license key on a verified transaction.completed event
POST /api/billing/polar/checkout	None	Seller-side: creates a Polar checkout session, returns its redirect URL
POST /api/billing/polar/webhook	Polar signature	Seller-side: issues a license key on a verified order.paid event

Upload Response

A successful upload returns the tenant's full, newly re-ranked lead list. One entry (trimmed for length):

200 – POST /API/LEADS/UPLOAD (EXCERPT)

```
{
  "id": "8f3a1c9d2b7e",
  "company_name": "Northwind Data",
  "domain": "northwinddata.com",
  "industry": "SaaS",
  "employee_count": 340,
  "revenue_usd": 41000000,
  "tech_stack": ["AWS", "Salesforce", "Snowflake"],
  "is_hiring": true,
  "enrichment_source": "apollo",
  "fit_score": 82.5,
  "score_breakdown": {
    "industry_match": 20.0,
    "company_size_fit": 20.0,
    "revenue_fit": 15.0,
    "tech_stack_match": 10.0,
    "geography_fit": 10.0,
    "title_seniority": 3.0,
    "hiring_signal": 10.0
  },
  "conversion_likelihood": 78.0,
  "llm_rationale": "Strong tech-stack overlap and active hiring suggest near-term budge",
  "combined_score": 80.7,
  "bucket": "hot",
  "outreach_draft": null,
  "crm_pushed": false
}
```

An unlicensed upload past the trial limit is capped rather than dropped — the response still succeeds, with the truncation communicated through response headers:

RESPONSE HEADERS, TRIAL-CAPPED UPLOAD

```
X-Trial-Limited-Rows: 10
X-Trial-Total-Rows: 214
```

License Status Variants

GET /API/LICENSE


```
// Licensed
{ "licensed": true, "customer_email": "buyer@acme.com", "plan": "monthly", "expires_at"

// Trial, still active
{ "licensed": false, "reason": "trial", "customer_email": null, "plan": null,
  "trial_days_left": 1.4, "trial_uploads_left": 6 }

// Trial exhausted (time or upload count, whichever came first)
{ "licensed": false, "reason": "trial_expired", "customer_email": null, "plan": null,
  "trial_days_left": null, "trial_uploads_left": null }

// A key is set but fails signature/parse verification
{ "licensed": false, "reason": "invalid", "customer_email": null, "plan": null,
  "trial_days_left": null, "trial_uploads_left": null }
```

Status Codes in Use

CODE	MEANING IN THIS API
400	Malformed/unparseable file, missing required columns, empty file, row cap exceeded
401	Bearer key present but doesn't match any tenant; wrong login credentials
402	Default tenant, unlicensed, trial window or upload count exhausted
404	Lead not found (or not owned by the requesting tenant)
409	Signup email already has an account
413	Uploaded file exceeds <code>MAX_UPLOAD_SIZE_MB</code>
429	Rate limit exceeded (slowapi's handler)
502 / 503	A configured payment/licensing dependency is unreachable or not configured

Tenant, Licensing & Access Control Model

Access States

STATE	WHO	GATED BY THIS DEPLOYMENT'S LICENSE
Default tenant	Single self-hosted buyer, no <code>Authorization</code> header sent	Yes — this is the buyer of the software itself
Self-serve tenant	A seller's own customer, signed up via <code>/api/accounts/signup</code>	Never — they're a customer of whoever runs the deployment, not a buyer of it
Seller-provisioned tenant	Manually created via <code>scripts/create_tenant.py</code> , API-key only, no login of its own	Never , same reasoning

License States

Offline Ed25519 verification (`licensing.py`) distinguishes four states so the UI never conflates "never purchased" with "already paid, something's wrong":

STATE	MEANING
<code>NONE</code>	No <code>LICENSE_KEY</code> set at all — a genuine, still-evaluating trial
<code>INVALID</code>	A key is set but is malformed or fails signature verification
<code>EXPIRED</code>	Signature is valid, but past its <code>expires_at</code>
<code>VALID</code>	Signature valid, not expired — full access

A buyer whose key is `INVALID` or `EXPIRED` is shown language distinct from a never-purchased trial — the former reads as "your payment may not have gone through," which risks a double-charge if shown to someone who already paid.

Trial Gate Rules

Two independent limits bound the free trial for the default tenant — whichever is hit first ends it:

- `TRIAL_DAYS` (default `3`) — the clock starts on this deployment's first request that ever checks it, persisted in `app_meta` , and is never reset by a restart.
- `TRIAL_MAX_UPLOADS` (default `10`) — total `/api/leads/upload` calls allowed unlicensed, also persisted and restart-proof.
- `TRIAL_MAX_LEADS_PER_UPLOAD` (default `10`) — each unlicensed upload call is truncated to this many rows, so a prospect can judge scoring quality on a real sample without full free use of a large list.
- `LICENSE_REQUIRED` (default `false`) — when set, skips the grace window entirely and 402s immediately with no valid key.

Access Control Requirements

REQUIREMENT	ENFORCEMENT
No <code>Authorization</code> header	Resolves to the default tenant — zero-config by design
Invalid/unknown Bearer key	<code>401</code> ; never silently falls back to the default tenant
Cross-tenant reads	Structurally impossible — every storage query filters on <code>tenant_id</code> ; a foreign lead ID resolves to <code>404</code> , not another tenant's data
Self-serve tenant licensing	Never gated by the deployment owner's own license — only the default tenant is
Paid-feature availability	Determined server-side from <code>verify_license()</code> — never a client-supplied flag
Churn scoring & health/license endpoints	Never license-gated, by design — free evaluation surface and operational status

Frontend Application Architecture

The React client is a single screen: upload, review, act. It is not treated as an authorization boundary — every action it offers is re-validated by FastAPI.

COMPONENT	BEHAVIOR
Upload panel	Drag-and-drop or click-to-browse CSV/XLSX upload; shows a scoring-in-progress state, a trial-truncation notice when applicable, and inline checkout buttons the moment a trial-expired upload is rejected
Leads table	Ranked, filterable by bucket (All / Hot / Warm / Cold); selecting a row opens the detail drawer
Lead detail drawer	Firmographics, a per-criterion score-breakdown bar chart (each bar traceable back to the exact enriched field it was computed from), the LLM rationale, an outreach-draft generator (email/LinkedIn toggle), and a "Push to Salesforce" action
License banner	Shown only for the default workspace (self-serve tenants never see license/trial messaging, since the license belongs to whoever runs the deployment). Renders one of four states — licensed, trial, trial-expired, invalid — each with its own copy and, where relevant, Paddle/Polar buy buttons
Tenant switcher	Small, unobtrusive control (top right) for log in / sign up / paste-an-API-key / disconnect — invisible in effect for a single self-hosted buyer, since no key means the default tenant
/purchase-complete	Post-checkout landing page (client-side route, served by the FastAPI catch-all in production)
/reset-password	Consumes a reset-link token from the URL, sets a new password, signs the tenant back in with the rotated key

The client never talks to Claude, Apollo, or Salesforce directly — every third-party call happens server-side. The frontend only ever calls its own `/api/*` surface, same-origin in production.

Payment & Licensing Infrastructure

Entitlement Model

A license is a small JSON payload (customer email, plan, expiry) signed with an Ed25519 private key the seller keeps; the buyer's deployment ships only the corresponding *public* key, safe to commit since it can verify signatures but never create them. Verification is entirely offline — no network call, works air-gapped.

- Monthly licenses are issued valid for **35 days**; annual licenses for **380 days** — comfortably longer than one billing period each, to tolerate payment retry/dunning delays.
- An offline-verified key **cannot be actively revoked** once issued. This is precisely why the validity windows are short rather than perpetual: a cancelled or failed subscription simply lapses on its own at expiry.
- Every `transaction.completed` webhook (Paddle) or `order.paid` webhook (Polar) — fired for both the first payment and every renewal — re-issues a fresh key, so an active subscriber never notices the expiry window exists.

Paddle Integration

- Checkout is an in-page overlay (Paddle.js), not a backend-generated redirect URL — Paddle's transaction API requires a real HTTPS origin approved in its dashboard, which a local/dev backend can't provide; the overlay has no such requirement.
- `GET /api/billing/config` exposes the non-secret client token and price IDs the overlay needs.
- Webhook signatures use Paddle's `ts / h1` scheme — HMAC-SHA256 of `"{timestamp}:{raw body}"`, rejected if the timestamp is more than 300 seconds from now (bounding replay of a captured request).
- `transaction.completed` doesn't include the buyer's email directly — a verified webhook makes one authenticated follow-up call to Paddle's `GET /customers/{id}` to resolve it.
- `subscription.canceled` / `transaction.payment_failed` are logged only; no new key is issued, and the existing one simply lapses at its own expiry.

Polar Integration

- A second, independent merchant-of-record processor — not a Paddle fallback, but an alternative for sellers Paddle can't serve (Polar pays out via Stripe Connect Express, whose supported recipient countries are broader than Stripe's own direct-merchant list).
- Redirect checkout, not an overlay: `POST /api/billing/polar/checkout` creates the session server-side and hands back a URL to send the buyer to.
- Webhook verification follows the Standard Webhooks spec (`webhook-id` / `webhook-timestamp` / `webhook-signature` headers, HMAC-SHA256 of `"id.timestamp.body"`) — a different scheme from Paddle's, supporting key rotation via multiple candidate signatures in one header.
- License issuance itself (`_issue_and_deliver`) is shared code with the Paddle path — only checkout initiation and webhook verification differ between the two processors.

License Delivery

On a verified webhook, a license key is signed, appended to `licensing/issued_licenses.jsonl`, and emailed — SendGrid first if configured (better deliverability/analytics at scale), otherwise plain SMTP. With neither configured, the key is still issued and logged; the seller sends it by hand from the JSONL file.

11

Open Product & Technical Decisions

Unlike a product still being designed, most of CRM Scoring's early open questions are already resolved in code. What remains is a mix of deliberate out-of-scope calls and known, documented trade-offs:

ITEM	STATUS	NOTES
Custom ML conversion model	OUT OF SCOPE – V1	Would need labeled historical won/lost outcomes; documented as a possible third scoring layer alongside rules + LLM
Outbound send automation	OUT OF SCOPE – BY DESIGN	Drafts are for human review/send only, never auto-sent, to avoid compliance/spam risk
CRM targets beyond Salesforce	SINGLE TARGET TODAY	<code>crm: Literal["salesforce"]</code> — extending <code>push_to_crm()</code> to HubSpot/Pipedrive is additive, not architectural
Enrichment providers beyond Apollo	SINGLE PROVIDER TODAY	<code>enrich_lead()</code> already isolates the live/mock switch behind one function
Multi-tenant concurrency at scale	SQLITE BY DEFAULT	Fine for one self-hosted buyer; a seller running shared multi-tenant volume should set <code>DATABASE_URL</code> to Postgres
License revocation	KNOWN LIMITATION – BY DESIGN	An offline key can't be actively revoked; validity windows (35/380 days) are kept short specifically so a lapsed subscription self-expires
OAuth / SSO login	NOT IMPLEMENTED	Self-serve accounts are email + password only today
Session-per-device	KNOWN LIMITATION – BY DESIGN	Login rotates the tenant's single API key rather than issuing per-device tokens; a second-device login signs the first out

User Journeys

12.1 – Self-Hosted Buyer Journey

Deploy the app

./run-prod.sh or Docker — no keys required to start



Upload a lead file

Trial clock starts on this first checked request



Review scored, ranked leads

Up to 10 rows per call, up to 10 calls, up to 3 days — whichever ends first



Trial ends

License banner shows Paddle / Polar checkout



Purchase → webhook verified

Signed license key emailed and logged



Set LICENSE_KEY in .env

Verified fully offline on every request — full, uncapped access

12.2 – Self-Serve Tenant Journey

Sign up (name, email, password)

Tenant provisioned; API key returned once



Key stored client-side

Sent as Authorization: Bearer on every request



Upload, score, act

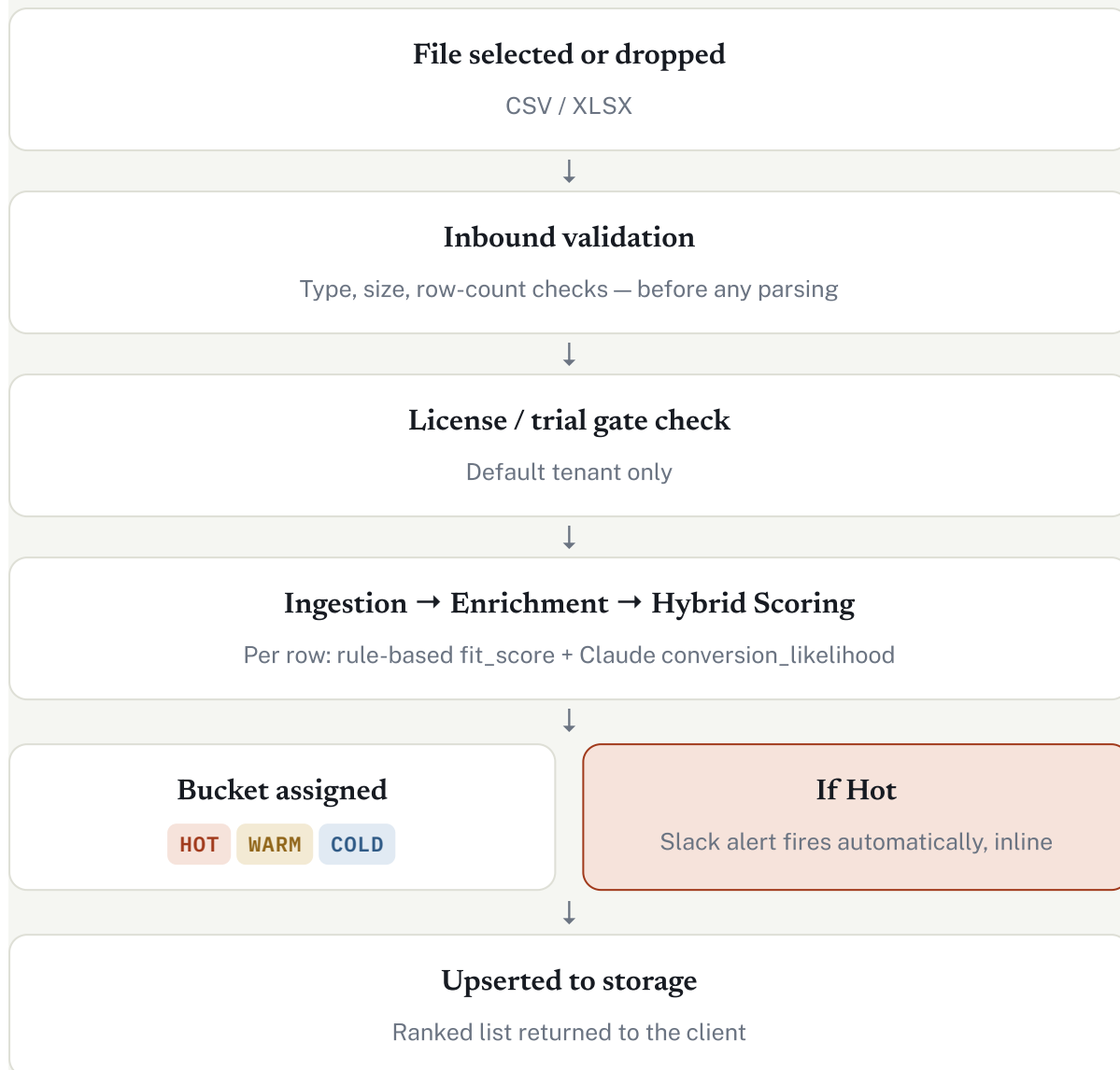
Full pipeline access — never license-gated



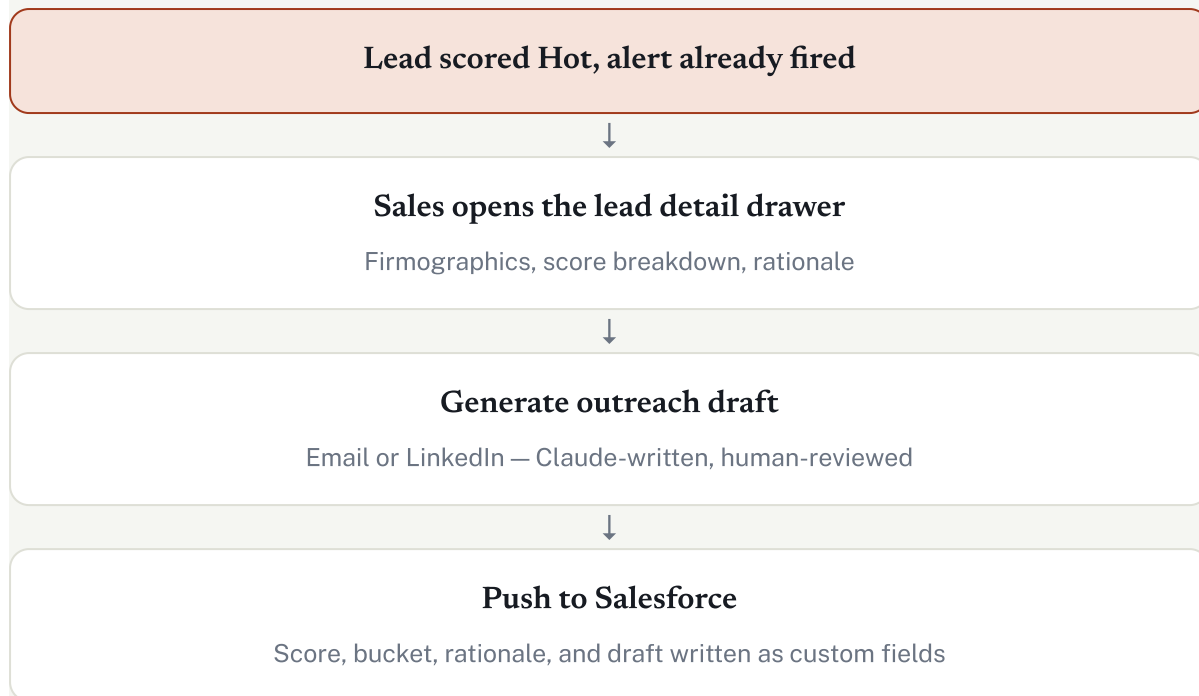
Log in later / another device

Fresh key issued, previous one invalidated

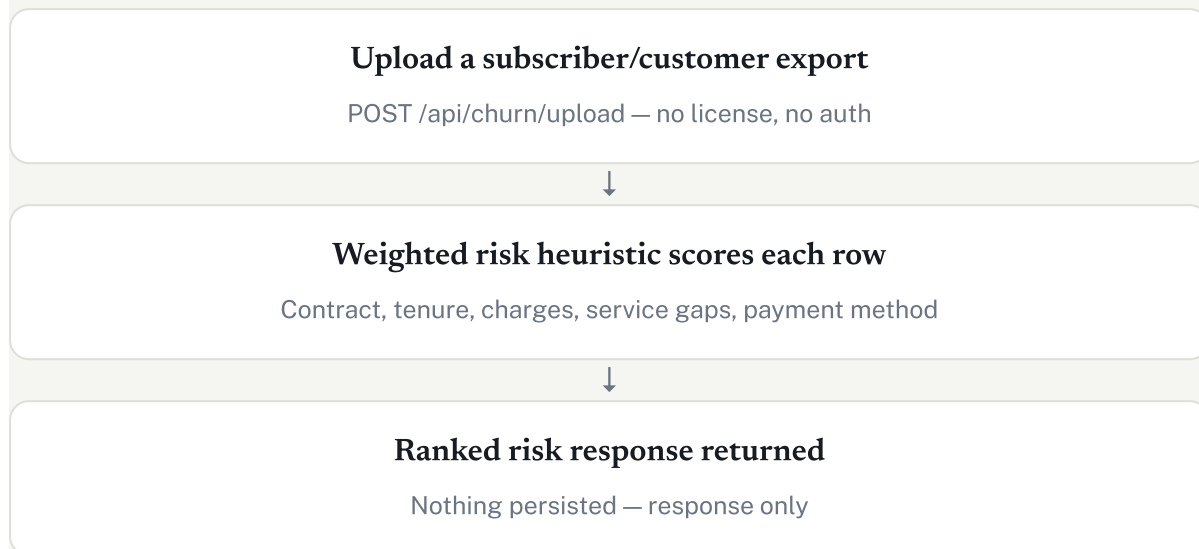
12.3 – Lead Upload & Scoring Journey



12.4 – Hot-Lead Action Journey



12.5 – Churn-Risk Upload Journey



12.6 – Password Reset Journey

"Forgot password?" → email submitted



Same response either way

No enumeration of registered emails



Reset link emailed (if the account exists)

Single-use token, 30-minute default TTL



New password set

Token deleted on use, valid or not



API key rotated

Tenant signed in with the new key

System Diagrams

Architecture Diagram

React SPA (Vite / Tailwind)

Upload · leads table · detail drawer · license banner



FastAPI Gateway

CORS · security headers · rate limits · tenant resolution



Upload Validation

Type / size / row-count caps



Lead Intelligence Pipeline

Ingestion → Enrichment → Hybrid Scoring → Actions

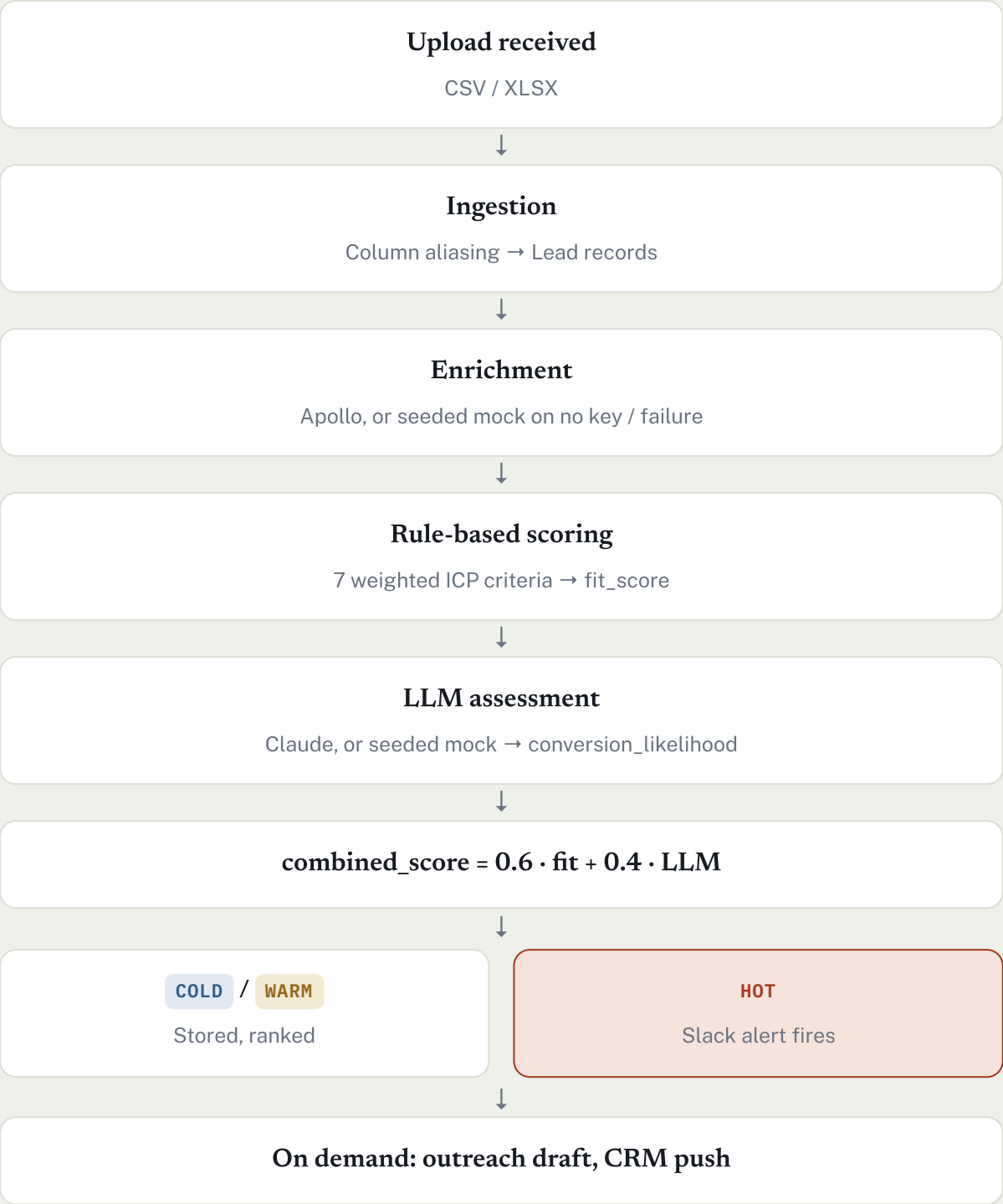


SQLite / Postgres

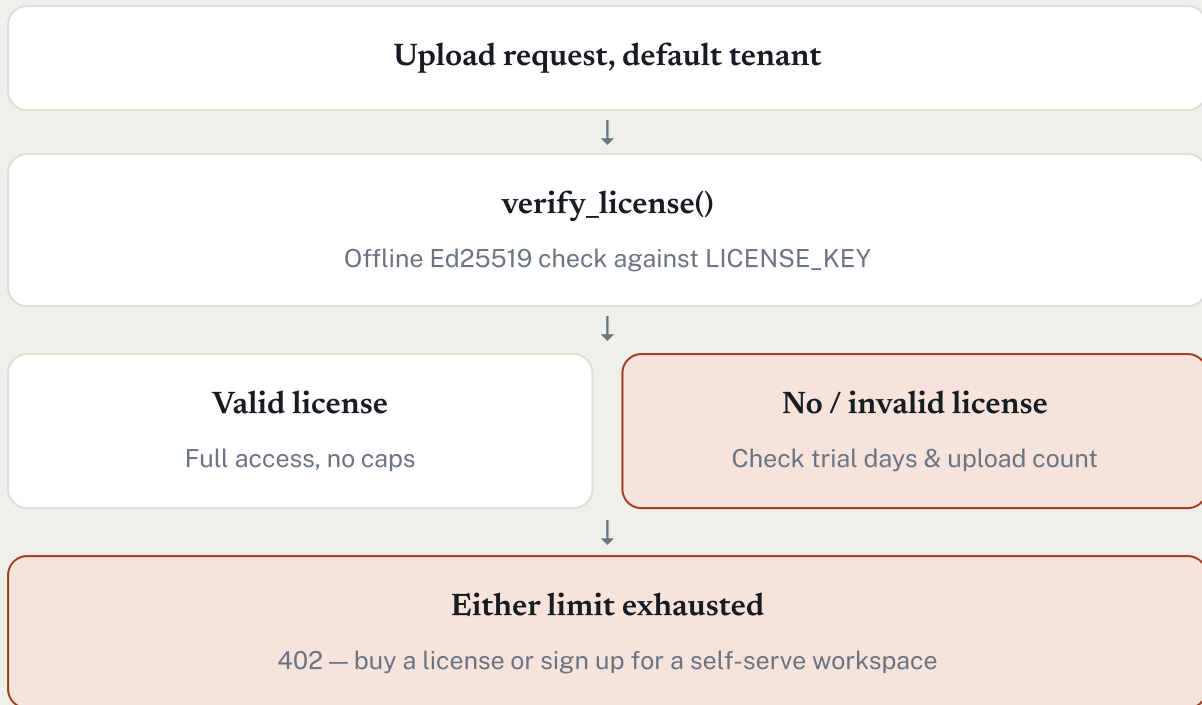
leads · alerts · tenants · password_resets · app_meta

Paddle / Polar webhooks call directly into `/api/billing/*`, independent of the lead pipeline – seller-side only.

Pipeline Flow Diagram



Trial & Licensing Flow



A self-serve tenant never enters this check at all — see §8.